SET NOCOUNT ON

Şəbəkə Yükü, TDS Paketlər

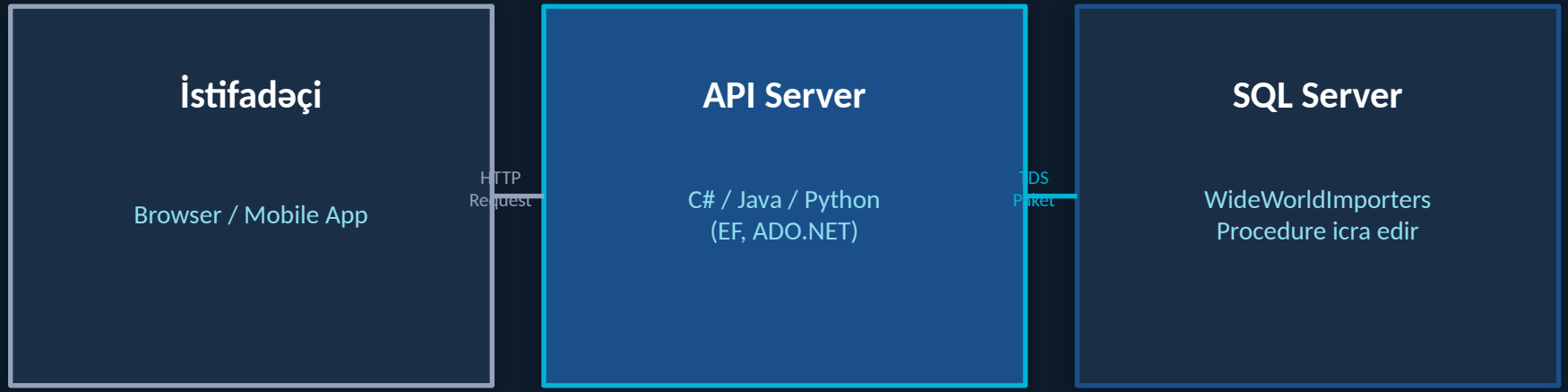
Production mühitdə API ↔ SQL Server arasında nə baş verir?

TDS Protokol | Connection Pool | ADO.NET | WideWorldImporters

```
SET NOCOUNT ON;

-- production
```

Production Arxitekturası: Kim Client-dir?



Əsas Məqam:

SSMS development-də client-dir. Production-da əsl client **API-dir (C#/Java/Python).** TDS paketlər **SQL Server ↔ API** şəbəkəsindən keçir — SSMS-in bununla əlaqəsi yoxdur.

TDS Protokolu: Paket axışı

Şəbəkə Yükü: Riyazi Hesab

Ssenari: 1 prosedurda 4 DML + 1 SELECT | Eyni anda 1000 request/saniyə

NOCOUNT OFF

5,000

paket / saniyə

`1000 req × 5 paket`

NOCOUNT ON

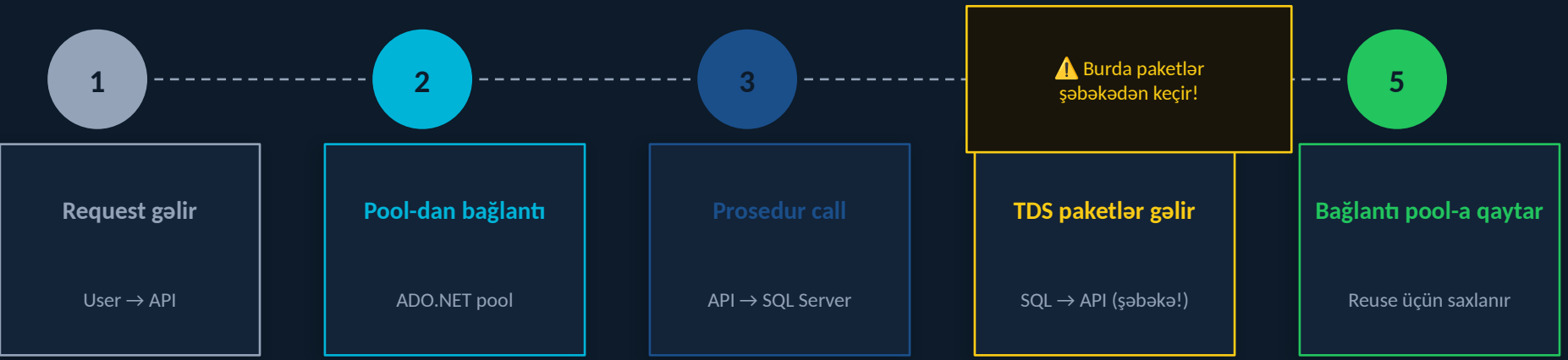
1,000

paket / saniyə

`1000 req × 1 paket`

💡 **4,000 lazımsız paket azaldıldı (80% azalma)**

Connection Pool ilə Tam Axış



SET NOCOUNT ON olmadan:

Step 4-də hər əmr üçün ayrıca TDS paketi gəlir. API bu paketləri alır, heç istifadə etmədən atar. Şəbəkə bandwidth-i boşuna xərclənir.

SET NOCOUNT ON ilə:

Step 4-də yalnız SELECT nəticəsi gəlir. DML əmrlərdən heç bir paket göndərilmir. Şəbəkə effektiv istifadə olunur.

Entity Framework / ADO.NET-də Vəziyyət

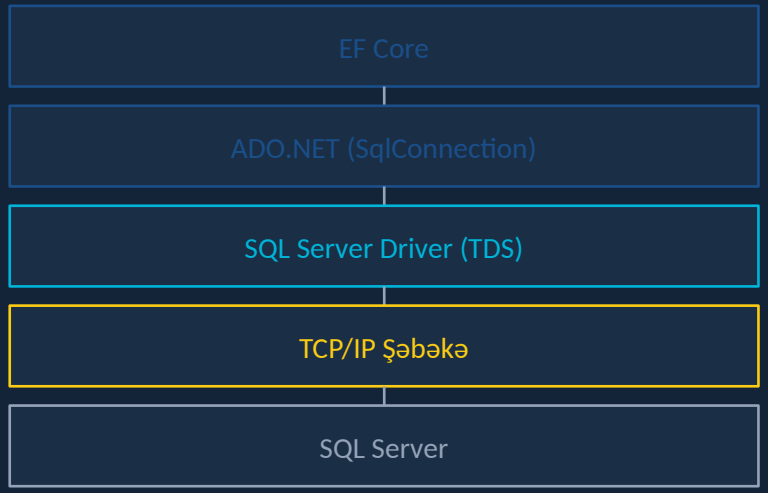
C# — EF ilə Call

```
using var ctx = new AppDbContext();

// EF stored procedure call
var orders = await ctx
  .Database
  .SqlQuery<OrderResult>(
    $"EXEC SifarisYarat
      {customerId},
      {stockId},
      {qty}")
  .ToListAsync();

// Altda ADO.NET işləyir!
// TDS paketlər:
//  NOCOUNT OFF → 5 paket
//  NOCOUNT ON  → 1 paket
```

Altda Nə Baş Verir?



⚡ EF istifadə etsən də, altda eyni TDS protokolu işləyir.

EF → ADO.NET → TDS Driver → Şəbəkə → SQL Server | Hər layer-də SET NOCOUNT ON effekti eynidir

WideWorldImporters: Canlı Nümunə

SifarisYarat Proseduru

```
CREATE PROCEDURE SifarisYarat
  @MusteriID INT, @StokID INT, @Miqdar INT
AS BEGIN
  SET NOCOUNT ON;  -- ← BURDA!
  DECLARE @YeniID TABLE (ID INT);

  INSERT INTO Sales.Orders (...)
  OUTPUT INSERTED.OrderID
  INTO @YeniID(ID)   -- Trigger-safe!
  VALUES (...);

  DECLARE @OrderID INT;
  SELECT @OrderID = ID FROM @YeniID;

  INSERT INTO Sales.OrderLines (...)
  SELECT @OrderID, ... FROM
    Warehouse.StockItems
    WHERE StockItemID = @StokID;

  SELECT o.OrderID, c.CustomerName,
    si.StockItemName, ol.Quantity
  FROM Sales.Orders o ...;
END;
```

Nəticə Müqayisəsi

NOCOUNT OFF

```
(1 row affected)
(1 row affected)
OrderID | CustomerName
73596   | Tailspin Toys
```

NOCOUNT ON

```
OrderID | CustomerName
73596   | Tailspin Toys
```

@@ROWCOUNT hər iki halda düzgün işləyir

OUTPUT INSERTED trigger-safe ID alma üsulu

Mühit Fərqləri və Xülasə

Mühit	Şəbəkə Gecikmə	NOCOUNT Effekti
Localhost (dev)	~0 ms	Hiss olunmaz
LAN (ofis şəbəkəsi)	1-5 ms	Orta
WAN / Internet	20-100 ms	Böyük ✅
1000+ req/saniyə	Yüksək yük	Kritik ✅

Əsas Qaydalar:

- ✅ Bütün stored procedure-lərdə `SET NOCOUNT ON` yazın
- ✅ `SCOPE_IDENTITY()` yox, `OUTPUT INSERTED` istifadə edin (trigger-safe)
- ✅ `@@ROWCOUNT` heç vaxt təsirlənmir — rahat istifadə edin
- ✅ Production-da client = API-dir (C#/Java), SSMS yox